

Data Science 101: Key Learning Points

One-page summary of the bootcamp, built around the car price (regression) and breast cancer (classification) assessments

1. The life cycle

Problem → **Collect** → **Clean** → **Explore** → **Features** → **Model** → **Evaluate** → **Deploy**

Hook: "Pretty Cats Can Eat Fish, Making Everyone Delighted"

2. Pick the problem type

Target is...	Type	Example
a number	Regression	Car price
a category	Classification	Malignant / benign
no target	Clustering	Customer groups

Regression = "how much?" · Classification = "which one?"

3. Tools

Jupyter: run code in cells, see output instantly.

NumPy: fast arrays, one data type, maths on everything at once. **Pandas**: tables (DataFrames).

Matplotlib/Seaborn: charts. **scikit-learn**: models, splitting, scaling, metrics.

Python basics: `s[::-1]` reverses a string · `lst[-1]` gets the last item · `import pandas as pd`

4. Inspect & clean first

- `head()` look · `info()` types & missing · `describe()` stats · `unique()` categories
- Missing values: replace placeholders like "?" with NaN, fill with mean (median if outliers)
- Drop useless columns: IDs (car_ID), text with too many values (CarName, 147 unique)

5. Explore (EDA)

Understand the data before modelling: patterns, outliers, relationships. Heatmap, scatter plot, pairplot, histogram.

Correlation runs from -1 to +1. Strong if beyond ± 0.7 , weak if near 0. **Not causation**.

Car data: engine size +0.87, curb weight +0.84, horsepower +0.81 with price. MPG $\approx$ -0.70 (efficient cars are cheaper).

Multicollinearity: two inputs carry the same info (city vs highway MPG, 0.97), so the model can't separate their effects. Drop or combine one.

6. Prepare the features

1. **Dummies** (`get_dummies`): text → 0/1 columns. Not 1/2/3, which invents a fake order and size.
2. **X / y**: X = inputs, y = target (price)
3. **Train/test split** 80/20, `random_state` for repeatable results
4. **Scale** (StandardScaler): `fit_transform` on train, `transform` only on test

Data leakage: learning anything from test data inflates results. "Don't study from the exam paper."

7. The modelling pattern

Create → `.fit(X_train, y_train)` → `.predict(X_test)` → **score**

Linear regression fits the best straight line by minimising squared errors (OLS).

8. Regression metrics

Metric	Meaning	Car model
R ²	% of variation explained (0-1)	0.86
RMSE	Typical error in real units	$\approx$ 3,284 (~25% of avg 13,277)

R² is **not accuracy**. Accuracy is only for classification. Sanity-check outputs: one prediction was **-1,954**. A straight line has no floor. Fix with `log(price)` or a tree model.

9. Classification models

Model	Idea	Acc.
Logistic regression	Boundary line + probability	96.5%
KNN	Vote of nearest neighbours (needs scaling)	95.6%
Decision tree	Yes/no questions, overfits easily	93.9%

Breast cancer labels: **0 = malignant**, **1 = benign**.

Built into scikit-learn (`load_breast_cancer()`).

10. Confusion matrix & metrics

Rows = actual, columns = predicted. TP / TN = right · FP = false alarm · FN = **miss**.

Precision: when it says yes, is it right? · **Recall**: of the real yeses, how many caught? · **F1**: balance.

Accuracy can lie. KNN missed 5 cancers vs 2 for logistic regression. When misses are costly, judge on recall.

11. Tuning & overfitting

Hyperparameters are settings you choose (`max_depth`, `C`, `n_neighbors`). **GridSearchCV** tries every combo with **5-fold cross-validation**: train on 4 parts, validate on 1, rotate, average.

Overfitting: memorises training data. High train score, low test score. Fix: simpler model, regularisation, more data, cross-validation.

12. How to answer an instructor

- Name the term, then finish with **"...because..."**
- Answer each item in a list separately
- Quote your numbers and say if they're good
- Not sure? Think out loud. Reasoning earns marks.